1. sort()

Syntax:

Ascending

```
sort(v.begin(), v.end());
```

Descending

```
sort(v.begin(), v.end(), greater<int>());
```

Sort array

```
sort(arr, arr+n);
```

Returns:

➡️ **Nothing (void)**

What it does:

Sorts the vector/array in ascending order.

---

2. binary_search()

Syntax:

```
binary_search(v.begin(), v.end(), x);
```

Returns:

➡️ **bool**
(true if x exists, false otherwise)

---

3. lower_bound()

Syntax:

```
auto it = lower_bound(v.begin(), v.end(), x);
```

Returns:

➡️ **Iterator** to first element **≥ x**

Convert to index:

```
int idx = it - v.begin();
```

---

4. upper_bound()

Syntax:

```
auto it = upper_bound(v.begin(), v.end(), x);
```

Returns:

➡️ **Iterator** to first element **> x**

Convert to index:

```
int idx = it - v.begin();
```

---

5. reverse()

Syntax:

```
reverse(v.begin(), v.end());
```

Returns:

➡️ **Nothing (void)**

What it does:

Reverses the order of elements.

---

6. max_element()

Syntax:

```
auto it = max_element(v.begin(), v.end());
```

Returns:

➡️ **Iterator** to the maximum element

Get value:

```
int mx = *it;
```

---

7. min_element()

Syntax:

```
auto it = min_element(v.begin(), v.end());
```

Returns:

➡️ **Iterator** to the minimum element

Get value:

```
int mn = *it;
```

---

8. count()

Syntax:

```
int c = count(v.begin(), v.end(), x);
```

Returns:

➡️ **int** (number of times x appears)

---

9. accumulate()

Include <numeric>.

Syntax:

```
int sum = accumulate(v.begin(), v.end(), 0);
```

Returns:

➡️ **sum of all elements** (int or long long)

---

10. find()

Syntax:

```
auto it = find(v.begin(), v.end(), x);
```

Returns:

➡️ **Iterator**

- if found → points to the element
- if not found → equals v.end()

---

11. erase()

Erase by index:

```
v.erase(v.begin() + i);
```

Returns:

➡️ Iterator to the next element after deletion

---

12. unique()

Syntax:

```
auto it = unique(v.begin(), v.end());
```

Returns:

➡️ Iterator to the new end (after removing adjacent duplicates)

Note: It doesn't resize vector. Must use with erase().

---

13. priority_queue

Max-heap:

```
priority_queue<int> pq;
```

Returns nothing (it's a container).

Methods:

```
pq.push(x);  // insert
pq.pop();    // remove top
pq.top();    // returns largest
pq.empty();  // bool
pq.size();   // int
```

---

14. Min-heap

Syntax:

priority_queue<int, vector<int>, greater<int>> pq;

Top returns smallest element.

---

15. map

Syntax:

map<int,int> mp;

mp[key] = value;

What it returns:

➡️ [] returns a **reference** to the mapped value.

---

find in map:

auto it = mp.find(key);

Returns iterator:

- found → it->first, it->second
- not found → mp.end()

---

16. unordered_map

Syntax:

unordered_map<int,int> mp;

mp[key]++;

Returns:
➡️ same key/value behavior as map
(But hashing, faster average O(1))

---

17. set

Syntax:

set<int> s;

s.insert(x);

Returns:

➡️ insert() returns a pair:
(iterator, bool insertedSuccessfully)

Summary Table (Quick Revision)

STL Function	Returns	Meaning
sort	void	Sorts container
reverse	void	Reverses order
binary_search	bool	Checks existence
lower_bound	iterator	First ≥ x
upper_bound	iterator	First > x
max_element	iterator	Max value
min_element	iterator	Min value
count	int	No. of occurrences
accumulate	int	Sum
find	iterator	Finds element
erase	iterator	Next position
unique	iterator	New end
priority_queue	container	Heap structure
map / unordered_map	ref / value	Key-value pairs
set	pair	Insert result